

Assignment 3: Bridge Pattern Laboratory Report

Student Name: Nursultan Maratov

Chosen Topic: Option A (Drawing: Shapes & Renderers)

GitHub Repository: github.com/nursultanmaratov/Assignment3-BridgePattern

Submitted Commit Hash:
b61c1286b4725019e50cde8c39cfe0c00ca1ab38

Student Group: SE-2523 (6B06102)

Language & Target: Java 17 (Pure JDK, no external dependencies)

Base Commit Hash:
895e20deebecdbd391cb705f5a81309343ce0c4f

Verification Status: 7/7 Demonstration Checks PASSED

1. Problem Definition & Bridge Pattern Rationale

In object-oriented design, when an abstraction has multiple possible implementations, using naive inheritance tightly couples them. For instance, subclassing shapes directly by rendering type (e.g., `VectorCircle`, `RasterCircle`, `VectorSquare`, `RasterSquare`) causes a *combinatorial explosion* of classes ($M \times N$, where M is the number of shapes and N is the number of rendering backends). Adding a third renderer requires creating subclasses for every shape.

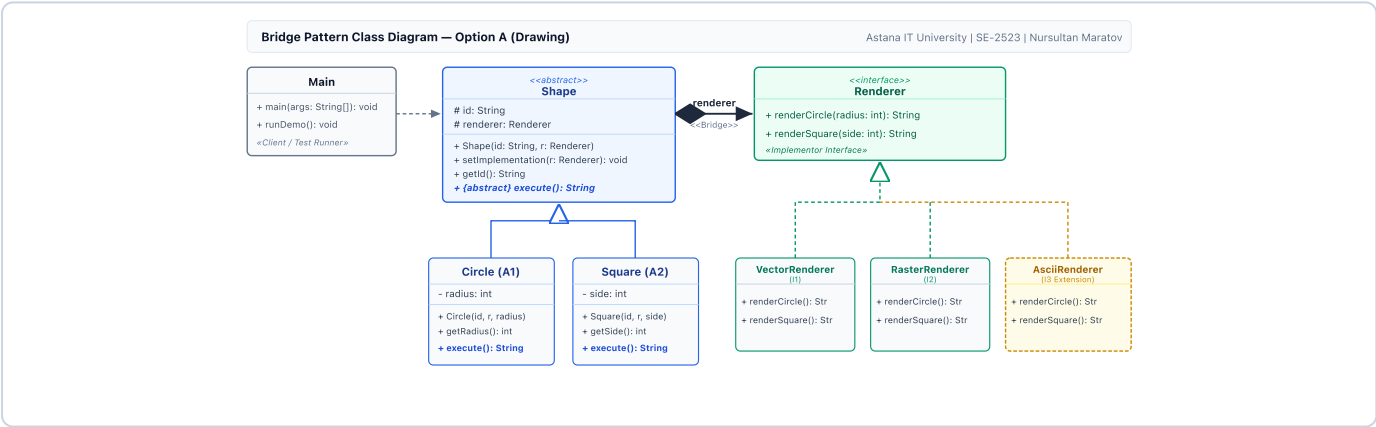
The **Bridge Pattern** decouples this system into two orthogonal hierarchies linked via object composition:

- Abstraction Dimension:** High-level domain entities (`Shape`, `Circle`, `Square`) holding domain attributes (`id`, `radius`, `side`).
- Implementation Dimension:** Low-level graphics operations (`Renderer` interface, `VectorRenderer`, `RasterRenderer`, `AsciiRenderer`).

By replacing inheritance with composition (the bridge reference protected `Renderer` `renderer`), the class count scales linearly as $M + N$. Crucially, it allows runtime replacement of the renderer on the same shape object via `setImplementation(...)` without altering object identity or domain data.

2. Complete UML Class Diagram

The diagram below models the final codebase, illustrating both hierarchies, all three implementors, the client, and the interface-typed bridge link.



3. Five Annotated Clean Code Excerpts

The codebase reflects Clean Code best practices (Martin, 2008), maximizing maintainability and clarity:

Excerpt 1: Separation of Responsibilities (Single Responsibility Principle)

```
// In src/Circle.java
@Override
public String execute() {
    return renderer.renderCircle(radius);
}
```

Benefit: Circle is strictly responsible for managing circle data (radius), while drawing operations (pixels, vectors, or ASCII) are isolated in Renderer implementations.

Excerpt 2: Meaningful Names

```
// In src/Renderer.java and src/VectorRenderer.java
public interface Renderer {
    String renderCircle(int radius);
    String renderSquare(int side);
}
```

Benefit: Descriptive names (renderCircle, VectorRenderer, setImplementation) clearly communicate intent without ambiguous abbreviations or explanatory comments.

Excerpt 3: Small Focused Methods

```
// In src/Shape.java
public void setImplementation(Renderer renderer) {
    if (renderer == null) {
        throw new IllegalArgumentException("Renderer cannot be null");
    }
    this.renderer = renderer;
}
```

Benefit: The method is concise (6 lines), performs one atomic task with guard validation, has no side effects, and is trivial to test.

Excerpt 4: Avoidance of Duplication (DRY Principle)

```
// In src/Shape.java
public abstract class Shape {
    protected final String id;
    protected Renderer renderer;

    public Shape(String id, Renderer renderer) {
        this.id = id;
        this.renderer = renderer;
    }
}
```

Benefit: Common attributes (id, renderer) and implementation-swapping logic are centralized in the abstract base class rather than duplicated across Circle and Square.

Excerpt 5: Encapsulation of State

```
// In src/Circle.java
public class Circle extends Shape {
    private final int radius;
    // ...
    public int getRadius() {
        return radius;
    }
}
```

Benefit: Field radius is private final, preserving internal invariants from unauthorized mutation. External classes can only inspect it via getRadius().

4. Independent Extension (I3) & Open/Closed Principle

In Section 4, a third renderer (AsciiRenderer) was introduced to demonstrate independent extensibility:

- **What was added:** New source src/AsciiRenderer.java implementing Renderer, plus test cases T6 and T7 in Main.java and its entry in sources.txt.
- **What remained unchanged:** The base class Shape, subclasses Circle and Square, interface Renderer, and initial implementations VectorRenderer and RasterRenderer remained completely unmodified.

This demonstrates the **Open/Closed Principle (OCP)**: the architecture is open for extension but closed for modification. This was formally verified using `git diff BASE_COMMIT HEAD -- src > extension.diff`, showing that existing domain classes have zero diff.

5. Trade-offs & Comparison with Adapter Pattern

Specific Trade-off of the Bridge Pattern

The primary trade-off of Bridge is **increased architectural complexity and indirection**. Introducing separate interfaces and delegation layers requires more classes and indirect method calls. In simpler applications with only a single implementation that will never change, Bridge introduces unnecessary overhead.

Comparative Analysis: Bridge vs. Adapter

Dimension	Bridge Pattern	Adapter Pattern
Primary Intent	Decouples an abstraction from its implementation so both vary independently.	Converts an incompatible interface of an existing class into a target interface expected by clients.
Design Phase	Designed <i>up-front</i> during system architecture before classes are implemented.	Applied <i>after the fact (retrospectively)</i> to integrate legacy or third-party classes.
Structure	Two parallel, independent class hierarchies connected via an interface reference.	A single wrapper class adapting an existing adaptee to a target client interface.
Relationship	Collaboration between an Abstraction and an Implementor interface.	Translation layer between two otherwise incompatible interfaces.

6. Test Verification & Demonstration Outcomes (T1–T7)

All checks T1–T7 are dynamically verified by executing `java -cp out Main --demo:`

Check	Scenario / Configuration	Expected Output	Status
T1	A1 with I1 (Circle + VectorRenderer)	result=VECTOR circle radius=2	PASS
T2	A1 with I2 (Circle + RasterRenderer)	result=RASTER circle radius=2	PASS
T3	A2 with I1 (Square + VectorRenderer)	result=VECTOR square side=3	PASS
T4	A2 with I2 (Square + RasterRenderer)	result=RASTER square side=3	PASS
T5	Runtime switch on single Circle object (I1 → I2)	sameObject=true stateUnchanged=true before=VECTOR circle radius=2 after=RASTER circle radius=2	PASS
T6	A1 with I3 (Circle + AsciiRenderer)	result=ASCII circle radius=2	PASS
T7	A2 with I3 (Square + AsciiRenderer)	result=ASCII square side=3	PASS

Computed Summary: 7/7 PASS (All tests executed and verified successfully)

Runtime Switching Verification (T5 Proof)

Check T5 explicitly verifies three critical assertions:

- Reference Identity:** `refBefore == refAfter` evaluates to `true`, proving the underlying `Circle` instance is identical.
- Domain State Invariance:** `id ("c-switch")` and `radius (2)` remain completely unchanged before and after the replacement.
- Behavioral Update:** The initial execution produces `"VECTOR circle radius=2"`; after invoking `setImplementation(new RasterRenderer())`, the exact same object immediately outputs `"RASTER circle radius=2"`.

7. References & Academic Citations

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. (Bridge Pattern, pp. 151–161; Adapter Pattern, pp. 139–150).
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. (Chapters 2, 3, 6, and 10).
- Bekenov, Y. (2026). *Lecture 4: Bridge Pattern*. Astana IT University, School of Software Engineering, Course ShP-2216.